

Aegis

AI-Assisted Vaccine & Therapeutic Discovery

A Technical Overview

1 The Problem, How Vaccine Discovery Actually Works Today

When a researcher is trying to develop a vaccine or therapeutic, they're essentially doing three things in a loop: reading papers to understand the pathogen, proposing candidate molecules or antigens, and then scoring those candidates against a list of criteria before deciding what to test in a lab.

The reading part alone can take months, PubMed has 36 million papers and growing. The scoring part is subjective, inconsistent across teams, and heavily dependent on whoever happens to be in the room. And the whole process is linear: you propose, you score, you test, you fail, you start over. There's no system that evolves its proposals based on what it already tried, no system that cites exactly which paper informed a decision, and no system that does all of this in one place in real time.

That's the gap Aegis fills.

2 What Aegis Is

Aegis is an AI-assisted research tool that takes your disease biology papers, builds a searchable knowledge base from them, and then runs an iterative simulation that proposes, scores, and evolves vaccine or therapeutic candidates, showing you the entire reasoning process in real time as a navigable graph.

Think of it as giving a research team an AI collaborator that has read everything you've given it, never forgets a detail, scores every candidate the same way every time, and shows its work.

3 The Tech Stack

The frontend is React with TypeScript, built with Vite, styled with Tailwind, and animated with Framer Motion, nothing exotic, just fast and clean. The backend is FastAPI in Python, which handles everything from PDF processing to running the simulation loop asynchronously.

The database is PostgreSQL via Supabase, extended with the pgvector plugin for storing and searching embedding vectors, the reason pgvector over a dedicated vector database like Pinecone is that it keeps everything in one place: your user data, your paper chunks, your candidates, and your RLS policies, all in the same database with no sync layer to break.

For AI, the system uses two models: OpenAI's `text-embedding-3-small` to convert text into 1536-

dimensional vectors, and Anthropic's Claude 3.5 Sonnet for all generation and scoring, Sonnet specifically because it's fast enough to run an iteration in about a minute without sacrificing reasoning quality.

The frontend talks to the backend over a WebSocket for real-time updates, proxied cleanly through Vite so there are zero CORS issues.

4 Where the Data Comes From

There are two sources. First, the user uploads their own PDFs directly, these are the papers their team has already curated, their own experimental results, whatever they consider authoritative.

Second, Aegis has a built-in PubMed discovery tool: you type in a disease name, and the system uses Claude to generate three optimized search queries (one focused on virology, one on pathogenesis, one on epidemiology), fires them against the NCBI E-utilities API, deduplicates the results, and then has Claude rank the returned papers by relevance with a one-sentence explanation of why each one matters.

If a paper has an open-access PDF on PubMed Central, you can import it directly into your project in one click, it gets downloaded, parsed, chunked, and embedded automatically. The NCBI integration is rate-limited (3 requests/second without an API key, 10 with) and has exponential backoff built in, because NCBI will block you if you hammer it.

5 The Brain: The RAG System

This is the most important part of the system to understand, so it's worth going through step by step.

When you upload a PDF, Aegis doesn't just store the file. It reads every page using PyMuPDF, then breaks the full text into overlapping chunks of 800 tokens with a 200-token overlap, the overlap is critical because a key sentence sitting at the boundary of two chunks shouldn't be missed just because of where the window fell.

Each chunk gets sent to OpenAI's embedding model, which converts it into a 1536-dimensional vector, a point in high-dimensional space where semantic similarity maps to geometric proximity. Those vectors get stored in PostgreSQL alongside the original text and metadata (which paper, which page, which chunk index).

When the simulation runs, before it proposes any candidates it needs to know what the literature says. So it embeds the current search query, which starts as the disease profile and goal, and evolves later, and runs a cosine similarity search against all stored chunk vectors. Cosine similarity measures the angle between two vectors: a score of 1 means identical direction (same meaning), 0 means orthogonal (unrelated). The system retrieves the 15 most semantically relevant chunks from your papers, not keyword matches, not the most recent papers, but the chunks that are most meaningfully close to what the simulation is currently thinking about.

5.1 What Makes It Adaptive

After the first few iterations, the search query isn't just the disease anymore. It incorporates the mechanisms of the top 3 candidates from the previous iteration. So the RAG system starts steering

its own retrieval, if the simulation is converging on mRNA antigen approaches, it starts pulling chunks about mRNA delivery, lipid nanoparticles, and immunogenicity for that class. The retrieval evolves with the reasoning, which means later iterations are informed by much more targeted evidence than early ones.

The reason this matters over just feeding all the papers into a context window is threefold: first, context windows are finite and noisy, 50 papers dumped in at once means the model is pattern-matching across hundreds of pages of text with no hierarchy. Second, retrieval forces the system to be selective, only the most relevant evidence makes it into each call, which means the model reasons more precisely. Third, it scales, you can have 200 papers in the system and the retrieval cost stays constant.

6 The Simulation Engine, Iterative Discovery

Each iteration of the simulation runs four steps in sequence.

- **RAG retrieval**, as described above.
- **Candidate generation**: Claude is given the disease profile, goal, any constraints you set, the retrieved evidence chunks, and the top 5 candidates from the previous iteration, and asked to propose 4 new candidates, structured as name, mechanism, components, rationale, and parent (which prior candidate it evolved from, if any).
- **Scoring**: each candidate is scored independently across 7 dimensions, immunogenicity, safety, efficacy, breadth of protection, manufacturability, stability, and novelty, each weighted (immunogenicity and safety each carry 20%, the rest between 10–15%).
- **Ranking and broadcast**: the candidates are ranked, stored, and broadcast over WebSocket to the frontend in real time.

The parent-child relationship is intentional: it lets you see on the graph exactly how a candidate evolved across iterations, which proposals died, and which lineages the AI found most promising. It's the evolutionary metaphor made literal.

The loop runs until it hits the max iterations you set, or until convergence, defined as the top score varying by less than 1% across 3 consecutive iterations. Convergence detection exists because once the system stops finding meaningfully better candidates, burning more tokens and time on it is wasteful.

7 Why the LLM Doesn't Hallucinate

There are four specific mechanisms working together here, not just prompt engineering.

Tool use for all outputs. Claude never returns free-form text that gets parsed. Every generation call and every scoring call uses Anthropic's tool-use feature, which forces the model to populate a strict JSON schema. If the schema says `raw_score` is a float between 0 and 1, it produces a float between 0 and 1, no narrative workarounds, no "approximately 0.7-ish" text.

Evidence citation baked into the score. When Claude scores a candidate, the schema requires

it to return the indices of the specific chunks it used as evidence. Those indices get mapped back to real chunk UUIDs in the database, which point to specific pages of specific papers. Every score is traceable to a source, not “Claude thought this was safe,” but “Claude scored safety 0.82, citing chunks from page 14 of Smith et al. 2023 and page 7 of the WHO technical brief.”

Double-blind generation and scoring. The model that generates a candidate and the model that scores it are separate API calls with separate system prompts. The scorer never sees the generator’s reasoning, which means it can’t rationalize a high score based on the generator’s confidence. This is the same principle as peer review, the author doesn’t grade their own paper.

Calibrated scoring prompts. LLMs have a well-known tendency to cluster scores around 0.65–0.75, what you might call the polite zone. The scoring system prompt explicitly addresses this with defined anchors: what a 0.9 safety score looks like versus a 0.5 versus a 0.2, with concrete examples. It’s not foolproof, but it measurably reduces score compression.

8 Pattern Recognition Across Uploaded Files

This is what the embedding space enables that keyword search never could. When you upload 10 papers about different pathogens, the system doesn’t know in advance that paper 3 and paper 7 both discuss spike protein conformational change. But when the simulation queries for “antigen structural stability,” the cosine similarity search finds both chunks because they’re geometrically close in the vector space, their semantic content is related even if they use different terminology.

This is how the system surfaces cross-paper signals: shared mechanisms described with different vocabulary, analogous approaches from different disease areas, contradictory evidence about the same antigen from different research groups. The model then has to reason across those signals within a single context window, which is where the actual pattern synthesis happens.

9 Real-Time Visualization

As the simulation runs, every candidate that gets scored and ranked is broadcast over WebSocket and rendered as a node in a graph. The x-axis is iteration, the y-axis is rank within that iteration, and edges connect parent candidates to their descendants. Node color encodes score: bright teal for strong candidates (above 0.6), muted for weaker ones.

You can click any node to open the full detail panel, mechanism, rationale, all 7 scored dimensions with Claude’s explanations. What you’re watching is the AI’s search topology in real time: where it explored, what it abandoned, what it kept evolving. It’s not a dashboard of outputs, it’s a map of reasoning.

10 Security & Data Isolation

Every user’s data is isolated at the database level using Supabase’s Row-Level Security. Every table has policies that enforce `auth.uid() = user_id` on every read and write, meaning even if someone got a valid JWT, they can only ever see their own papers, simulations, and candidates.

The backend uses a service-role key that bypasses RLS so it can write on behalf of users during background processing, but that key never touches the browser, only the anon key does, which is

fully gated by RLS. API keys (Anthropic, OpenAI, NCBI) are server-side only.

11 Structural Validation (Vaccine Mode)

When you're running in vaccine mode, there's an additional layer after the simulation completes: the structural validator maps each candidate's primary antigen to a real entry in the Protein Data Bank (PDB), a global repository of experimentally determined 3D protein structures.

It then benchmarks the candidate against 2–3 approved vaccines for the same disease (BNT162b2, mRNA-1273, etc.) and returns a compatibility score with a narrative assessment. This grounds the AI's proposals in actual structural biology, a candidate might score well on immunogenicity in the simulation, but if its proposed antigen doesn't map to any known structure and has nothing in common with approved vaccines, that's a flag.

The system uses Claude to do this mapping via tool use, and the 3D structure itself is rendered in the frontend using 3Dmol.

12 Known Flaws & Limitations

No wet-lab validation. Aegis is a reasoning and retrieval tool. Everything it produces is a computational prediction. A high-scoring candidate still needs to be synthesized, tested in vitro, tested in animal models, and put through clinical trials. The system accelerates the hypothesis generation step, not the validation step.

Cost per simulation. Running a 10-iteration simulation costs roughly \$5–15 in Claude API calls, plus a smaller amount for OpenAI embeddings. That's manageable for a research team but adds up fast at scale, the scoring strategy (one Claude call per candidate per dimension set) is intentionally quality-over-cost.

Single-server event bus. The WebSocket event system uses an in-memory asyncio queue. That's fast and simple, but it means if you scale to multiple backend instances, events won't route correctly across servers. It's a known architectural limitation that would need Redis pub/sub or a similar message broker to fix.

JWT verification. The backend decodes JWTs without verifying the signature in the current implementation, relying on Supabase's infrastructure being the trust boundary. In a production hardened deployment you'd want full JWKS-based signature verification.

Cold-start problem. The RAG system is only as good as the papers you feed it. An empty project, no papers uploaded, will produce candidates based entirely on Claude's training data with no evidence grounding, which is exactly the hallucination scenario the system is designed to avoid. The paper suggestions feature exists specifically to solve this, but it still requires the user to actually import papers before simulating.

13 Where this goes next

There's still a lot of work ahead. Specifically around rigorous benchmarking and testing, since right now Aegis's scoring is internally consistent but hasn't been validated against real wet-lab outcomes or expert review. Closing that gap is the next major milestone.

I'm also aware that the system's ceiling is partly set by my own domain knowledge. The more I understand virology, immunology, and structural biology, the better I can design the scoring criteria, catch where the model's reasoning goes wrong, and push the system toward genuinely useful science rather than just plausible-sounding output. That's not a side note — it's the direction I'm actively building toward.

Aegis isn't a project I'm finishing. It's one I plan on growing for as long as I'm working at the intersection of computer science and medicine.

That's Aegis.